

Нейронная сеть на практике: Градиентный спуск

0

Daniel Jose

89

Введение

В предыдущей статье [«Нейронная сеть на практике: Пример реализации элемента XOR»](#) мы рассмотрели, как можно решить эту проблему, или, точнее, сам факт того, что один нейрон не способен научиться представлять логический элемент XOR или его инверсию — NXOR. Хотя один нейрон способен описывать различные случаи, думаю, уже стало вполне ясно, что существуют ситуации, с которыми он не может справиться, даже если на первый взгляд они кажутся довольно простыми.

И хотя на первый взгляд может показаться, что нейрон способен представлять эти же данные, бывают ситуации, когда этого просто не происходит. Именно в этом до сих пор и состояла цель: показать, что, вопреки тому, что многие говорят или хотят заставить вас поверить, бывают случаи, когда мы не можем добиться работоспособности решения лишь потому, что оно работает в простом и конкретном случае.

Таким образом, искусственный нейрон — далеко не нечто столь необычное, как многие полагают. Сам по себе это всего лишь простой математический расчёт. И не более того. Тем не менее, думаю, вы уже заметили, что, хотя мы и можем заставить искусственный нейрон представлять что-то, в некоторых случаях можем столкнуться с определёнными трудностями. Это связано с тем, что каждый раз, когда потребуется внести изменение, нам придётся каким-то образом модифицировать нейрон, чтобы он вёл себя должным образом и мог представлять определённые типы данных. Во многих случаях подобная ситуация делает проект совершенно нежизнеспособным.

Как вы, следящие за этими статьями о нейронных сетях, уже, наверное, заметили, окончательного решения не существует. Решения, которые охватывают большее или меньшее число случаев, существуют, но окончательного среди них нет. Это не мешает нам выбрать конкретное направление или попытаться реализовать что-то, что сделает использование искусственных нейронов применимым.

При изучении нейронных сетей, особенно в самом начале, возникает множество вопросов, которые могут в той или иной степени сбивать с толку. Я знаю это, потому что сам прошёл через тот же процесс, когда начал изучать программирование на C/C++. В то время эта тема не была так широко распространена и не была настолько на слуху. По правде говоря, об этом говорили немногие, и все они делали это довольно скептически.

В отличие от нынешней ситуации, когда средства массовой информации в целом, похоже, чрезмерно педалируют эту тему, утверждая, что нейронные сети — это то или иное. И, что ещё хуже, многие, особенно инвесторы, говорили вещи, далёкие от реальности. Я здесь не для того, чтобы судить кого-либо или разжигать споры. Я хочу лишь поделиться тем, что знаю, в как можно более доступной форме. Так у вас, по крайней мере, будет отправная точка, чтобы немного лучше разобраться в этой теме.

Что ж, в этой статье мы сделаем нечто немного необычное. Я знаю, что многие считают, будто мои статьи о нейронных сетях образуют серию, хотя на самом деле создать серию статей на эту тему невозможно. Можете мне поверить: как бы странно это ни звучало, создать серию статей о нейронных сетях практически невозможно. Это связано с тем, что нам снова и снова приходится возвращаться к

истокам. И причина проста: ЧЕЛОВЕЧЕСТВО НЕ ИМЕЕТ ЧЁТКОГО ПРЕДСТАВЛЕНИЯ О ТОМ, КАК РАЗРАБАТЫВАТЬ НЕЧТО ПОДОБНОЕ.

То, что я только что сказал, может показаться совершенно нелогичным, ведь, судя по всему, каждый день появляется что-то новое на эту тему. Однако на практике всё обстоит не совсем так. На самом деле происходит следующее: нам снова и снова приходится возвращаться к тому, что уже было сделано, и переосмысливать это, пытаясь решить какую-нибудь новую возникшую проблему.

В предыдущих статьях вы, вероятно, видели, что, хотя всё и работает, мы по-прежнему сталкиваемся с рядом проблем. Все эти проблемы уже решены или решаются прямо сейчас, так или иначе. Среди проблем, которые я могу здесь упомянуть и которые мы уже рассматривали в этих статьях, можно выделить следующую: чем больше переменных или параметров нам приходится обрабатывать, тем медленнее становится нейрон или нейронная сеть. Кроме того, по мере того как нам приходится обрабатывать всё больше и больше переменных, растут и темпы потребления вычислительных ресурсов. Ещё одна проблема заключается в том, что у нас нет большой гибкости в том, чтобы добавлять в нейрон большее или меньшее число параметров или переменных.

Эту проблему можно лучше понять следующим образом: начнём с одного входа в нейрон. Об этом говорилось в первых статьях, где была представлена первая схема нейрона. Но вскоре пришлось добавить новый вход. Итак, предыдущий код выглядел примерно так, как показано в следующем фрагменте:

```
//+-----+
double Cost(const double w)
{
    double err;

    err = 0;
    for (uint c = 0; c < nTrain; c++)
        err += MathPow((Train[c][0] * w) - Train[c][1], 2);

    return err / nTrain;
}
//+-----+
```

Вскоре это стало выглядеть так, как показано в следующем фрагменте:

```
//+-----+
double Cost(const double w0, const double w1, const double b, const double &Train[][])
{
    double err;

    err = 0;
    for (uint c = 0; c < nTrain; c++)
        err += MathPow(((Train[c][0] * w0) + (Train[c][1] * w1) + b) - Train[c][2], 2);

    return err / nTrain;
}
//+-----+
```

Обратите внимание, что код начал разрастаться и становиться несколько сложнее. Но если вы в это не верите, я хочу, чтобы вы задумались о следующем: каждый раз, когда в нейрон добавляется новая переменная или параметр, нам придётся изменять код. Поскольку каждый новый параметр представляет собой новый вход, если в какой-то момент нам потребуется использовать тысячу входов, нам придётся включить в код все эти тысячу переменных. И всё это вручную. Подумайте, какой огромный объём ручной работы это потребовало бы.

Ещё одна проблема связана с тем, как мы выполняем вычисления для настройки переменных. То есть с тем, каким образом нейрон или нейронная сеть будут представлять определённый набор данных. Возможно, именно этот вопрос имеет наибольший вес при решении о том, какую реализацию нейрона выбрать. Чтобы вам было понятнее, давайте возьмём за основу код, который мы рассматривали в предыдущей статье. Этот код показан в следующем фрагменте.

```
//+-----+
double Cost(const double w0, const double w1, const double b)
{
    double err;

    err = 0;
    for (uint c = 0; c < nTrain; c++)
        err += MathPow((macroSigmoid((Train[c][0] * w0) + (Train[c][1] * w1) + b) - Tra

    return err / nTrain;
}
//+-----+
double Cost_2(double &w0, double &w1, double &b)
{
    double err, ew0, ew1, eb;

    err = ew0 = ew1 = eb = 0;
    for (uint c = 0; c < nTrain; c++)
    {
        err += MathPow((macroSigmoid((Train[c][0] * w0) + (Train[c][1] * w1) + b) - Tra
        ew0 += MathPow((macroSigmoid((Train[c][0] * (w0 + eps)) + (Train[c][1] * w1) +
        ew1 += MathPow((macroSigmoid((Train[c][0] * w0) + (Train[c][1] * (w1 + eps)) +
        eb += MathPow((macroSigmoid((Train[c][0] * w0) + (Train[c][1] * w1) + (b + eps)

    }

    w0 -= (((ew0 - err) / eps) * eps);
    w1 -= (((ew1 - err) / eps) * eps);
    b  -= (((eb - err) / eps) * eps);

    return err / nTrain;
}
//+-----+
```

В нём мы адаптировали код так, чтобы выполнение было быстрее. Это подводит нас к первой проблеме, связанной с ростом вычислительных затрат по мере увеличения числа задач, которые нам нужно выполнить. Или, как многие говорят: чем больше вещей нам нужно анализировать, тем больше вычислительных ресурсов нам потребуется. Хотя обе функции, приведённые в предыдущем фрагменте, используют совершенно одинаковые вычисления, скорость их выполнения сильно различается. Это связано именно с тем, что первую функцию приходится вызывать много раз с течением времени. Вторая же вызывается гораздо реже, и именно в этом вся разница.

Кроме того, есть ещё один вопрос. Возможно, вы этого не заметили, но с самого начала весь код был сосредоточен на одной-единственной математической формуле. Эта формула — формула линейной регрессии. Да, мы использовали разные методологии для получения линейной регрессии. Но в конечном итоге все они свелись к использованию линейной регрессии.

Если линейная регрессия работает, зачем пробовать другую методику? Итак, здесь мы подходим к другому вопросу, связанному с нейронными сетями. Цель каждой методологии — создать математическое представление данных, которые мы используем. Если вы используете более быструю методику, вам понадобится меньше вычислительных ресурсов для получения тех же результатов.

В худшем случае мы можем прибегнуть к полному перебору, который, хотя и работает, совершенно нежизнеспособен. Это связано с тем, что полный перебор всегда требует гораздо больше вычислительных ресурсов, чем метод, основанный на математической формуле. Линейная регрессия работает во многих случаях. Однако по мере увеличения числа параметров она тоже начинает сталкиваться с той же проблемой, что и полный перебор. То есть для её выполнения в разумные сроки требуется всё больше вычислительных ресурсов.

Если вы посмотрите на формулу линейной регрессии, которую мы используем, то заметите нечто любопытное и интересное. И именно на этом мы и сосредоточимся в этой статье. Давайте теперь перейдём к новой теме, чтобы понять, что мы будем делать дальше.

И снова производные, но под новым названием

Чтобы максимально упростить всё, давайте вернёмся к самым базовым понятиям, чтобы вы могли проследить математическое рассуждение, которое мы будем использовать. Я не очень люблю приводить формулы и тому подобное, потому что зачастую это создает впечатление, будто всё гораздо сложнее, чем на самом деле. Но другого выбора нет. Чтобы объяснить это должным образом, нам нужно немного обратиться к математике. В большинстве случаев линейная регрессия вычисляется методом наименьших квадратов. То есть вы берёте ошибку, полученную в результате вычисления, и возводите её в квадрат. Складываете все полученные квадраты — и получаете общую ошибку. Именно это и отражает следующее уравнение.

$$Err = \sum_{n=1}^k (x_n w - y_n)^2$$

Здесь я свожу всё к самому базовому уровню. Следовательно, смещение здесь не учитывается. Используется только вес. Итак, значение k отражает общее количество данных, которые мы используем. Или, точнее, оно показывает, сколько данных у нас есть для построения линейной регрессии. Значение x соответствует данным, используемым при обучении, как и значение y . Разница заключается в том, что x обозначает входные данные, а y — ожидаемый результат. Единственное, что нам нужно изменить, — это вес, который соответствует значению w . Хорошо, это основа.

Это уравнение говорит нам следующее: если мы подставим произвольное значение w — а это единственное, что мы можем изменять, — то получим столь же произвольную ошибку. Если затем мы снова произвольно изменим значение w , то получим новую ошибку. Сравнив эти две ошибки, мы сможем понять, движемся ли мы в правильном направлении или в неправильном. Если мы движемся в правильном направлении, можно продолжать; если же в неправильном, нужно остановиться и изменить направление. Вот и всё. В этом же уравнении, приведённом выше, есть одна деталь, которая привлекает наше внимание: мы возводим значение в квадрат. И уже сам этот факт позволяет нам найти её производную. Именно здесь анализ начинает становиться по-настоящему интересным.

Когда это уравнение дифференцируют, оно получает новое название. Это объясняется довольно любопытной особенностью самого уравнения. Новое название: ГРАДИЕНТНЫЙ СПУСК. Скорее всего, вы уже слышали об этом градиентном спуске. Но знаете ли вы, откуда это взялось? Что ж, если ответ «нет», то теперь вы это знаете. Градиентный спуск — это уравнение, полученное путём дифференцирования линейной регрессии. Я не буду здесь показывать, как преобразовать линейную регрессию в градиентный спуск, поскольку это совершенно выходит за рамки данной статьи.

Если вам интересно узнать, как это делается, в конце статьи, в разделе ссылок, я приведу источник, где вы сможете изучить эту тему. Правда, будьте готовы к тому, что вам придётся столкнуться с немалым количеством математики. Объяснение, приведённое там, гораздо лучше того, которое я мог бы дать

здесь, поскольку наша цель — само программирование. При дифференцировании, то есть при преобразовании предыдущего уравнения, вы получите следующее уравнение.

$$Err = 2 \sum_{n=1}^k (x_n w - y_n) x_n$$

Это верно только для случая, когда учитывается один лишь вес. Но что, если мы также хотим использовать смещение? В таком случае у нас будет две разные функции, а не одна, как вы, возможно, ожидали. Эти две функции приведены ниже.

$$\frac{\partial Cost}{\partial w} = 2 \sum_{n=1}^k ((x_n w + b) - y_n) x_n$$

Здесь мы ищем минимум функции стоимости, варьируя только вес, основываясь только на весе. То есть мы изменяем вес и оставляем смещение неизменным, стремясь тем самым найти наименьшую стоимость для параметра веса. На следующем рисунке, напротив, мы пытаемся уменьшить функцию потерь, основываясь на смещении. То есть мы оставляем вес неизменным и изменяем только смещение.

$$\frac{\partial Cost}{\partial b} = 2 \sum_{n=1}^k ((x_n w + b) - y_n)$$

"Ой, подождите минутку. Единственное различие между ними заключается в конце уравнения?" Да, мой дорогой читатель. Чтобы вы поняли это и причину такого различия, предлагаю вам ознакомиться с материалами, которые я привожу в качестве справочной информации, или самостоятельно изучить, как в данном случае вычисляется производная. Так вы сможете глубже понять математические вопросы, связанные с этим градиентным спуском.

Хотя градиентный спуск — лишь один из многих методов уменьшения функции потерь, используемых для построения прямой, или уравнения, представляющего данные, я должен признать, что в настоящее время он является одним из наиболее широко применяемых. Тем не менее, существуют методы, которые в некоторых случаях оказываются более эффективными. В других случаях даже сам градиентный спуск может подвергаться новым математическим преобразованиям с целью создания ещё более совершенной методологии.

Именно по этой причине при изучении нейронных сетей встречается так много математических терминов. Дело не в том, что одна методология лучше или хуже другой. Дело в том, что мы, программисты, всегда ищем новые способы выполнения вычислений, но преследуем одну и ту же цель: снизить вычислительные затраты. То есть делать то же самое, но быстрее. Именно в этом заключается цель всех этих исследований в области математики, когда речь идёт о нейронных сетях.

Хорошо, а как всё это воплотить в коде? Именно это нас действительно интересует, и именно поэтому я и написал эту статью. Чтобы это увидеть, давайте перейдём к новой теме.

Программирование нейрона с помощью градиентного спуска

Здесь мы вернёмся к самому простому коду из всех. То есть мы не будем заниматься функциями активации, смещением, множественными входами и тому подобным. Давайте снова начнём с исходной системы: один вход и один выход, как будто мы раньше ничего не делали. Так вы сможете

действительно понять, как это следует реализовать. Если бы мы начали с чего-то более сложного, вы бы потратили много времени, пытаясь понять, почему это работает или почему не работает так, как вы ожидали. Итак, давайте снова начнём с нуля.

Для этого давайте посмотрим, каким был первый нейрон, который мы рассматривали в этих статьях о нейронных сетях. Полный код приведён ниже.

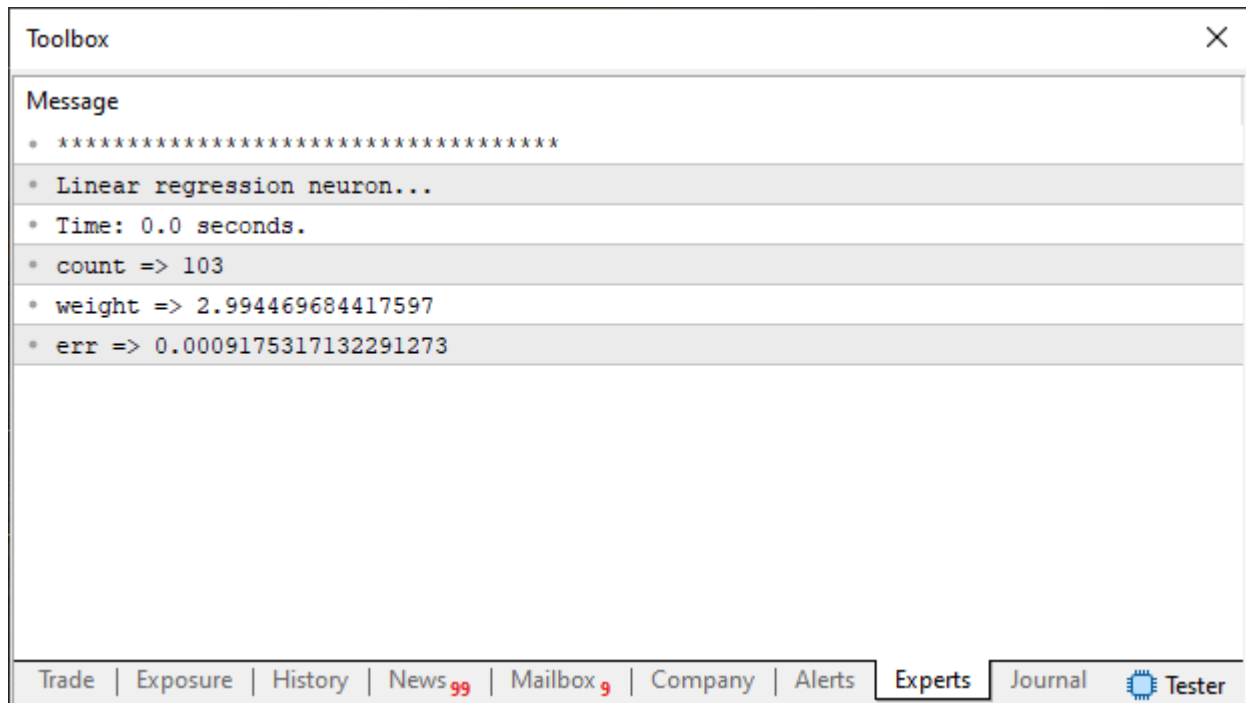
```
01. //+-----+
02. #property copyright "Daniel Jose"
03. //+-----+
04. #define PrintEx(A) Print(#A, " => ", A)
05. #define macroRandom (rand() / (double)SHORT_MAX)
06. //+-----+
07. double Train[][2] {
08.     {0, 0},
09.     {1, 3},
10.     {2, 6},
11.     {3, 9},
12.     {4, 12},
13. };
14. //+-----+
15. const uint nTrain = Train.Size() / 2;
16. const double epsilon = 1e-3;
17. //+-----+
18. double Cost(const double w)
19. {
20.     double err, factor;
21.
22.     err = 0;
23.     for (uint c = 0; c < nTrain; c++)
24.     {
25.         factor = Train[c][0] * w;
26.         err += MathPow(factor - Train[c][1], 2);
27.     }
28.
29.     return err;
30. }
31. //+-----+
32. void OnStart()
33. {
34.     double weight, err, e1;
35.     ulong it0, it1, count;
36.
37.     Print("*****");
38.     Print("Linear regression neuron...");
39.     MathSrand(512);
40.     weight = (double)macroRandom;
41.
42.     it0 = GetTickCount();
43.     for(count = 0; (count < ULONG_MAX) && ((err = Cost(weight)) > epsilon); count++)
44.     {
45.         e1 = (Cost(weight + epsilon) - err) / epsilon;
46.         weight -= (e1 * epsilon);
47.     }
48.     it1 = GetTickCount();
49.     Print("Time: ", (it1 - it0) / 1000.0, " seconds.");
50.     PrintEx(count);
```

```

51.     PrintEx(weight);
52.     PrintEx(err);
53. }
54. //+-----+

```

Конечно, я внес некоторые изменения в код, чтобы сделать его ещё проще и нагляднее, чтобы его могли легко понять даже те, кто не слишком разбирается в математике, но обладает минимальными знаниями в области программирования. Для этого не нужно ничего особенного; не обязательно быть мастером-джедаем в программировании, чтобы понять, что происходит. Кроме того, код, очень похожий на этот, уже объяснялся ранее. Идея и принцип работы, однако, остаются теми же. Обратите внимание, что в строке 38 я указываю, какая модель была использована для функции потерь. При выполнении этого кода в терминале MetaTrader 5 вы увидите следующее изображение:



Обратите внимание на данные на этом изображении, потому что я хочу, чтобы вы сравнили их с тем, что мы увидим в следующем коде. Хорошо, теперь у нас есть параметр для сравнения. Теперь давайте рассмотрим этот же нейрон или, точнее, это же обучение, но с использованием функции потерь, основанной на градиентном спуске. Полный код приведён ниже.

```

01. //+-----+
02. #property copyright "Daniel Jose"
03. //+-----+
04. #define PrintEx(A) Print(#A, " => ", A)
05. #define macroRandom (rand() / (double)SHORT_MAX)
06. //+-----+
07. double Train[][2] {
08.     {0, 0},
09.     {1, 3},
10.     {2, 6},
11.     {3, 9},
12.     {4, 12},
13. };
14. //+-----+
15. const uint nTrain = Train.Size() / 2;
16. const double epsilon = 1e-3;
17. //+-----+
18. double Cost(const double w)
19. {

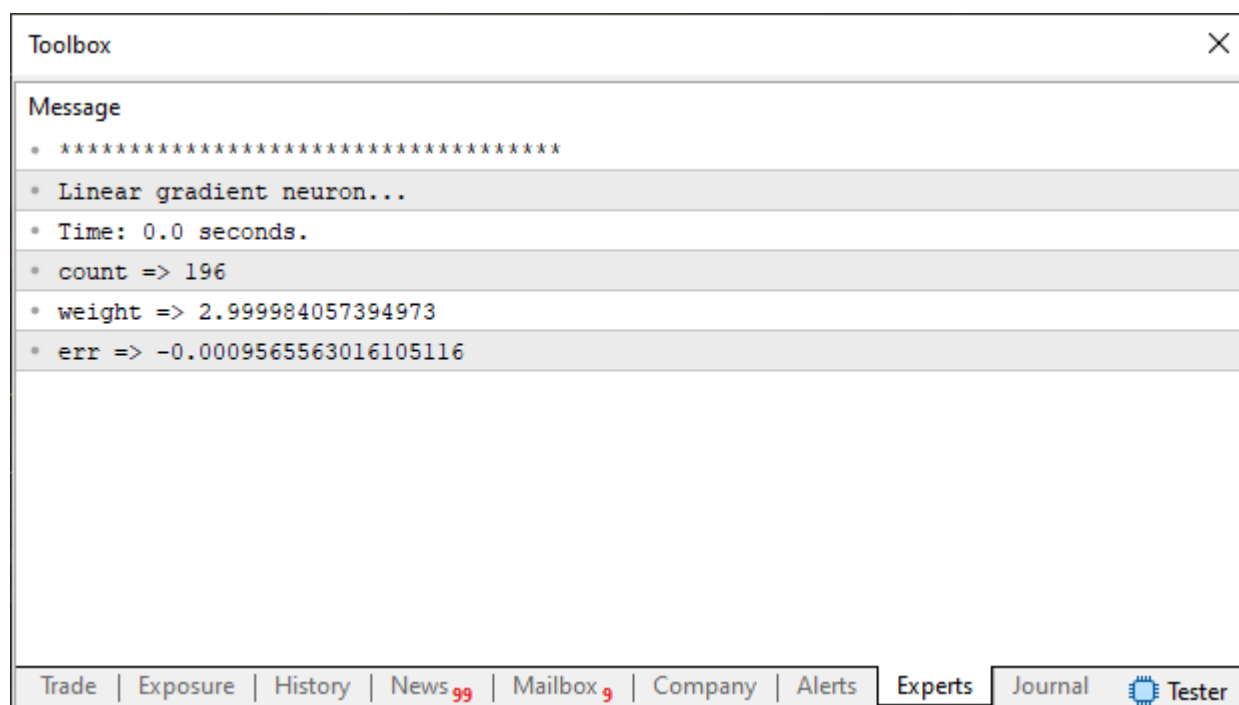
```

```

20.     double err, x, y;
21.
22.     err = 0;
23.     for (uint c = 0; c < nTrain; c++)
24.     {
25.         x = Train[c][0];
26.         y = Train[c][1];
27.         err += 2 *(x * w - y) * x;
28.     }
29.
30.     return err;
31. }
32. //+-----+
33. void OnStart()
34. {
35.     double weight, err;
36.     ulong it0, it1, count;
37.
38.     Print("*****");
39.     Print("Linear gradient neuron...");
40.     MathSrand(512);
41.     weight = (double)macroRandom;
42.
43.     it0 = GetTickCount();
44.     for(count = 0; (count < ULONG_MAX) && (MathAbs(err = Cost(weight)) > epsilon);
45.         weight -= (err * epsilon);
46.     it1 = GetTickCount();
47.     Print("Time: ", (it1 - it0) / 1000.0, " seconds.");
48.     PrintEx(count);
49.     PrintEx(weight);
50.     PrintEx(err);
51. }
52. //+-----+

```

При выполнении этого кода в терминале MetaTrader 5 вы увидите следующее изображение:



Теперь сравните оба изображения: полученное с помощью градиентного спуска и полученное с помощью линейной регрессии. Заметили какую-нибудь разницу? Ваш ответ "да", "нет" или "может быть"?

Что ж, чтобы вам не пришлось напрягаться, попытаюсь найти эту разницу, на следующем изображении оба результата показаны вместе. Так будет проще заметить разницу.

Toolbox		×
Message		
• *****		
• Linear regression neuron...		
• Time: 0.0 seconds.		
• count => 103		
• weight => 2.994469684417597		
• err => 0.0009175317132291273		
• *****		
• Linear gradient neuron...		
• Time: 0.0 seconds.		
• count => 196		
• weight => 2.999984057394973		
• err => -0.0009565563016105116		
Trade	Exposure	History
News ₉₉	Mailbox ₉	Company
Alerts	Experts	Journal
		 Tester

Ну, я не замечаю никакой разницы, разве что в счётчике. Что ж, мой дорогой читатель, если это ваш случай, взгляните ещё раз, не торопясь. Обратите внимание, что линейная регрессия, по-видимому, сработала быстрее. Однако полученное ею значение веса немного менее точное, чем значение, полученное с помощью градиентного спуска. Возможно, сейчас вы думаете: «Хорошо, но нам ведь нужно, чтобы функция потерь выполнялась как можно быстрее, не так ли?» Да, мой дорогой читатель, и я полностью согласен с этим замечанием. Именно поэтому я и показываю это сравнение.

Я хочу, чтобы вы не поддались заблуждению тех, кто утверждает или упорно настаивает на том, что градиентный спуск лучше линейной регрессии. Всё зависит от обстоятельств. И это зависит прежде всего от типа данных, которые мы используем для обучения нейрона. То есть от типа данных, которые мы используем для получения параметров уравнения. Эти значения призваны как можно точнее отражать данные из нашей базы данных. Метод, который считают лучшим, не всегда действительно оказывается лучшим. Каждый случай индивидуален.

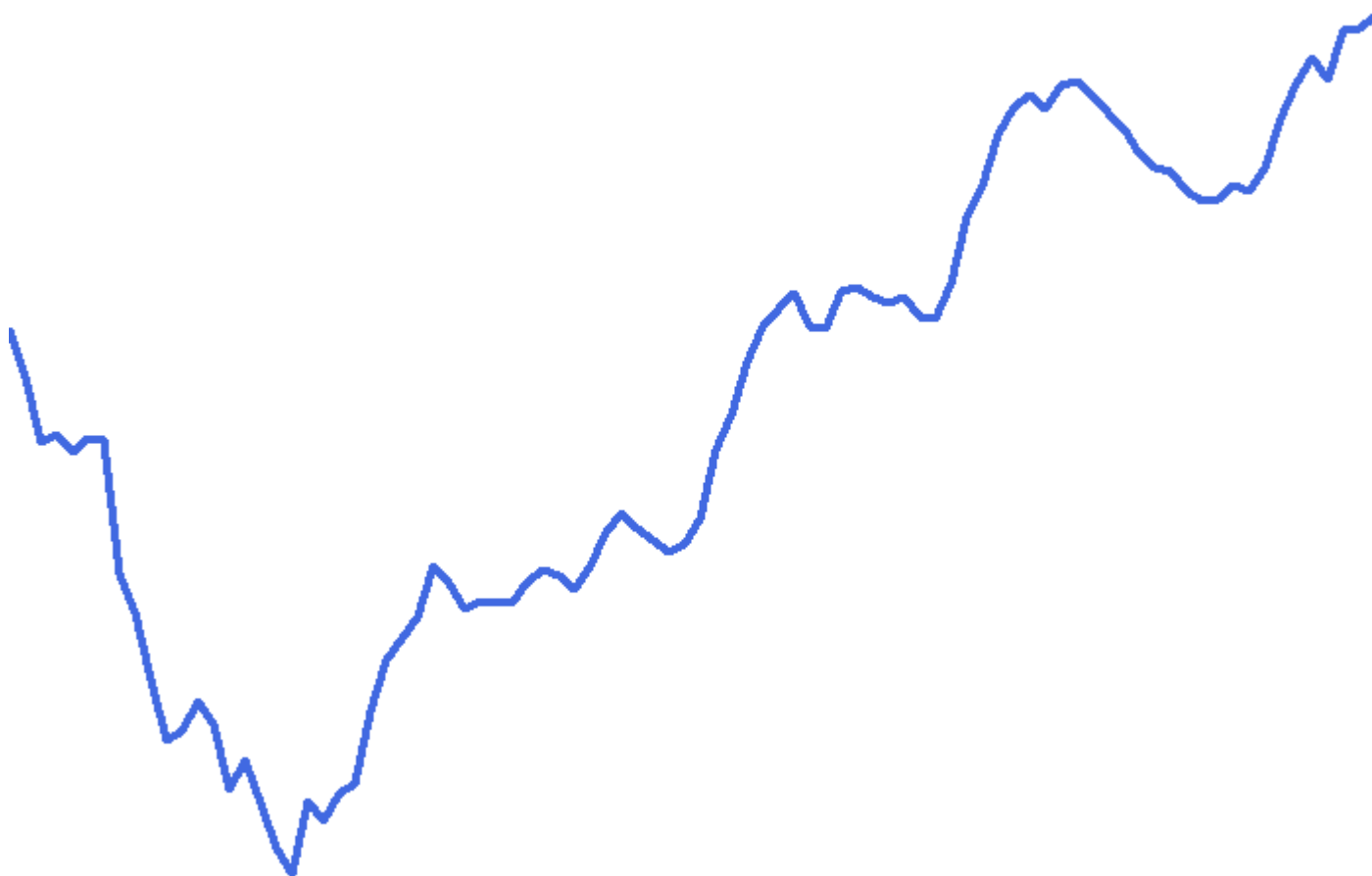
Обратите также внимание, что значение ошибки отличается. В случае линейной регрессии оно всегда будет положительным. В случае градиентного спуска, напротив, иногда оно может быть положительным, а иногда — отрицательным. Это связано с тем, что сама функция пытается найти наилучшую точку для уменьшения функции потерь. "Но подождите минутку. Как это? Я этого не понимаю. Разве полученный результат не должен всегда быть одинаковым, независимо от того, какой тип функции мы используем для поиска наилучшего представления данных?" Нет, мой дорогой читатель, и это ещё одна из ошибок или заблуждений, которые многие повторяют, когда речь заходит о нейронных сетях. Или, точнее, когда речь идет об обучении нейронной сети, будь то с использованием одного или нескольких нейронов. Чтобы объяснить это, давайте перейдем к новой теме. Так я смогу лучше разграничить эти понятия.

Что представляет собой столь часто упоминаемая «функция потерь»?

Величина, которую все называют «потерей», представляет собой ошибку функции или, точнее, ошибку представления, сгенерированного функцией, по сравнению с реальными данными, содержащимися в базе данных и использованными для построения этой функции. Хорошо, но в чём же тут проблема? Я никак не могу понять, почему база данных с одними и теми же данными может порождать столь разные функции. Разве они не должны быть одинаковыми, независимо от того, какой тип моделирования используется для построения представления данных из базы данных? Однозначно, ответить на этот вопрос не так-то просто. В любом случае, это будет зависеть от каждой конкретной ситуации.

Все приведенные мной примеры составлены так, чтобы быть максимально простыми. Поэтому во всех случаях общую функцию, описывающую обучающие данные, можно легко представить в виде параболы. То есть у нас есть единственная точка сходимости для системы. Таким образом, независимо от начальной точки, модели всегда будут сходиться к этой единственной точке. Однако в РЕАЛЬНЫХ условиях так бывает не всегда. Кривая, наилучшим образом описывающая данные из базы данных, может быть довольно сложным многочленом. Он может быть третьей степени или выше, и чем выше эта степень, тем больше влияние начальной точки при поиске наилучшего представления данных. То есть, даже если используемое моделирование не сможет сойтись к точке, где функции потерь будет минимальной, у нас всё равно сложится впечатление, что мы нашли наилучшую из возможных точек.

Показать подобные ситуации гораздо сложнее, чем кажется. Тем не менее, они происходят чаще, чем вы, мой дорогой читатель, могли бы себе представить. Чтобы немного лучше проиллюстрировать то, о чём я только что сказал, давайте представим следующее: предположим, что у вас есть база данных, всё содержимое которой можно представить с помощью математической функции. И при построении графика этой функции вы получаете график, показанный на следующем изображении:



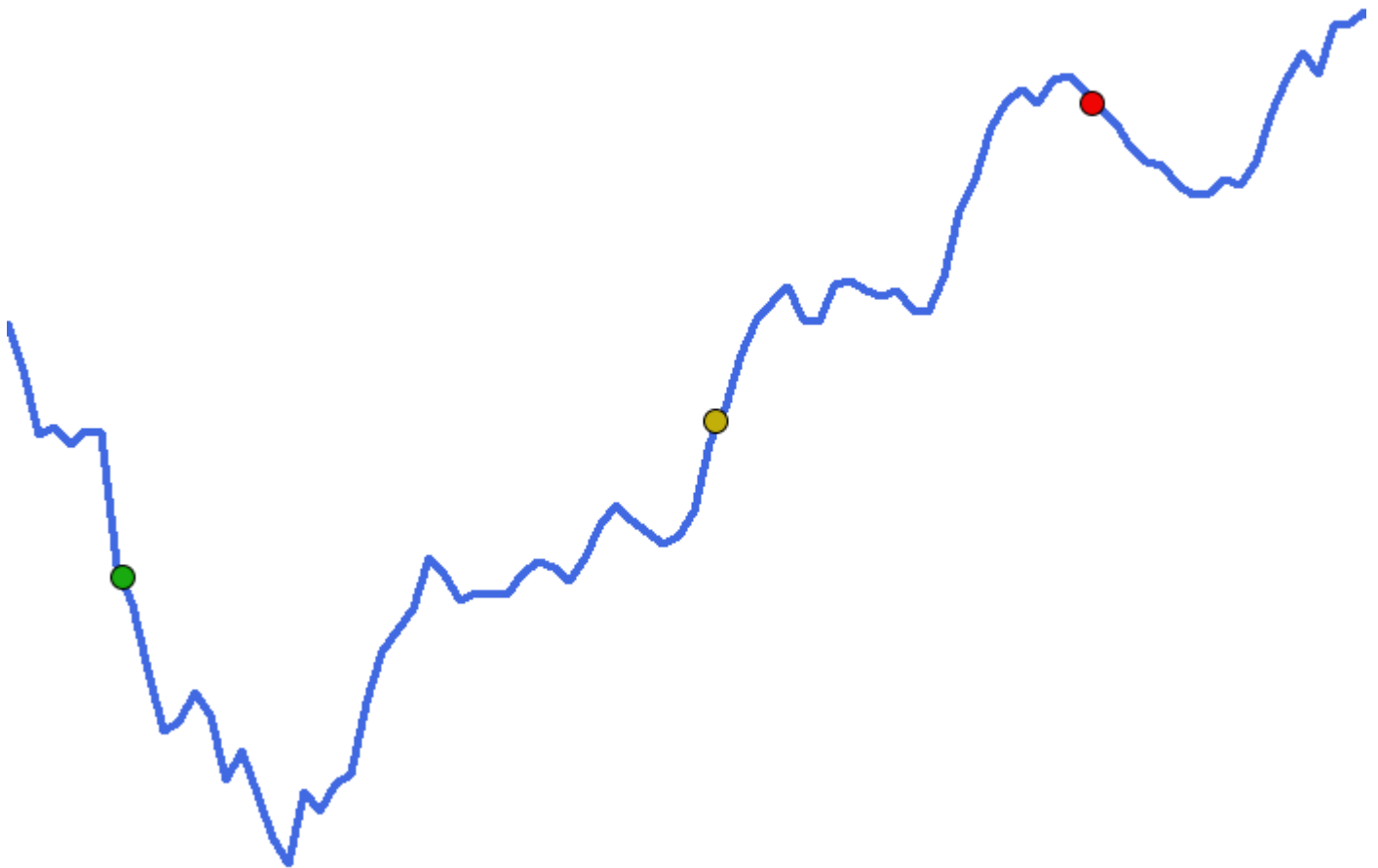
Мой вопрос таков: какая нейронная сеть лучше всего подходит для построения математического представления этой базы данных? Идея заключается в том, чтобы не потерять никаких данных из-за того, что график представления, созданного нейронной сетью, отличается от показанного выше. Хм, теперь у нас действительно проблема, не так ли? И именно в этой точке сосредоточены все исследования в области нейронных сетей. Всё, абсолютно всё, сводится к этому.

Вы можете подумать вот что: можно использовать линейную регрессию. Или, возможно, градиентный спуск. Или стохастический градиентный спуск. Или мы можем использовать полный перебор. Да, мой дорогой читатель. В худшем случае всё может свестись к полному перебору. Но подумайте о вычислительной мощности, необходимой для построения уравнения, представляющего эти данные. В наши дни это практически невыполнимо. Возможно, когда-нибудь квантовый компьютер действительно

появится. Но до тех пор давайте забудем о полном переборе. Нам нужно использовать другой метод. И вот тут-то и возникает проблема.

Очевидно, что на этой кривой есть точка минимума. Именно в этой точке функция будет иметь минимально возможную потерю, по крайней мере, судя по графику. Однако многие утверждают, что, пытаясь найти эту точку, мы на самом деле будем заниматься переобучением. Но это полная ерунда. Идея здесь в том, чтобы создать функцию, которая в идеале представляла бы все данные, содержащиеся в базе данных. Так откуда же взялась эта идея о переобучении? Дело в том, что эта идея возникает из-за того, что все хотят получить наилучшую функцию, но, найдя её, мы можем просто не суметь распознать элементы, которые находятся вне базы данных.

Кроме того, есть ещё одна причина: вычислительные затраты. Подумайте вот о чём. Любой метод моделирования, предназначенный для представления базы данных, требует одного или нескольких исходных данных. Когда все эти исходные данные обрабатываются, для функции вычисляется начальная потеря. И вот в чём суть. Где находится эта начальная точка? Итак, вернёмся к нашему рисунку и на этот раз добавим несколько точек на прямую.



Каждая из этих точек была создана совершенно случайным образом, точно так же, как мы всегда инициализируем нейрон или нейронную сеть. Однако они отражают начальную потерю, обусловленную именно этими случайными данными. А теперь начинается самое интересное. Какая из этих точек действительно достигнет точки с наименьшей потерей на этой кривой, показанной синим цветом? Что ж, это будет зависеть от того, как каждая точка будет искать эту точку с наименьшей потерей. В зависимости от того, как каждая из них выполняет поиск, может оказаться, что ни одна так и не достигнет этой точки наилучшей сходимости, то есть точки с наименьшей потерей. Это связано с тем, что на их пути встречаются и впадины, и вершины, и никто не знает наверняка, как преодолеть эти вершины. Они всегда ищут самую глубокую впадину, начиная с той вершины, на которой оказались изначально. Поэтому вопрос переобучения на самом деле не является главной проблемой. Настоящая

проблема заключается в следующем: сколько вычислительных ресурсов будет потрачено впустую на поиск самой глубокой впадины или точки с наименьшей потерей?

Заключительные замечания

В этой статье я постарался как можно проще и доступнее изложить один из самых спорных вопросов, возникающих при обсуждении нейронных сетей: как найти наилучшую возможную точку или наименьшую функцию потерь. Я показал разницу между линейной регрессией и градиентным спуском. Оба случая довольно просты и служат примером того, что не всегда путь, который кажется очевидным, действительно является лучшим.

В области исследования нейронных сетей ещё многое предстоит сделать. Так что, если вас это интересует, постарайтесь изучать эту тему спокойно и всегда ищите качественные материалы. Есть много плохо информированных людей, которые лишь повторяют заблуждения, которых сами даже не понимают. В приложении я привожу коды, использованные в этой статье.

Несмотря на всё сказанное здесь, мы ещё вернёмся к этой теме. Здесь я лишь слегка коснулся того, что на самом деле лежит в основе этого вопроса.

Ссылка

[Линейная регрессия и градиентный спуск: от нуля до магии](#)

Следующая статья >>

Daniel Jose

[Бразилия](#)

[52269](#)

Другие статьи автора

- [Нейронная сеть на практике: Пример реализации элемента XOR](#)
- [Репликация и моделирование рынка: Большой финал](#)
- [Моделирование рынка: В единстве — сила \(III\)](#)
- [Моделирование рынка: В единстве — сила \(II\)](#)
- [Моделирование рынка: В единстве — сила \(I\)](#)
- [Моделирование рынка: Position View \(XX\)](#)

[Подпишитесь](#) и узнавайте о популярных статьях и кодах